МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №5

по дисциплине: "Объектно-ориентированное программирование"

на тему: "Изучение механизмов исключений в языке C++"

Вариант 6

Выполнил студент группы 24ВВВ3:

Пяткин Р. С.

Принял:

Евсеева Ю. И.

Гудков А. А.

Пенза 2026

Лабораторное задание

1. Доработать шаблонный класс из предыдущей лабораторной работы таким образом, чтобы в программе осуществлялась генерация и обработка исключительных ситуаций.

   Исключительные ситуации генерируются:
   1) В конструкторе с параметром при попытке создать контейнер больше максимального размера;
   2) В операции []
   3) При попытке обратиться к элементу с номером меньше 0 или больше текущего размера контейнера ;
   4) В операции +
   5) При попытке добавить элемент с номером больше максимального размера;
   6) В операции –
   7) При попытке удалить элемент из пустого контейнера.

2. Создать программу, использующую механизм обработки исключительных ситуаций, позволяющую выполнять перевод вещественного числа из одной системы счисления в другую.

   Вариант 6: Перевод числа из тринадцатеричной системы счисления в восьмеричную.

Результат программы

```
=== Интерактивная проверка исключений класса Set ===
Введите максимальный размер множеств: 3
Введите размер первого множества: 4
Введите 4 элементов(а): 1 2 3 4
Исключение при вводе множества: Ошибка: превышен максимальный размер множества (add)
```

Рис. 1 — Исключение 1.

```
Текущие множества:
A: { 1, 2 }
B: { 1, 2, 3 }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 1
Введите индекс: 3

!!! Поймано исключение: Ошибка: индекс вне допустимого диапазона (operator[])
Программа завершена из-за исключения.
```

Рис. 2 — Исключение 2.

```
Текущие множества:
A: { 1, 2, 3 }
B: {  }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 1
Введите индекс: -1

!!! Поймано исключение: Ошибка: индекс вне допустимого диапазона (operator[])
Программа завершена из-за исключения.
```

Рис. 3 — Исключение 3.

```
Текущие множества:
A: { 5, 6 }
B: {  }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 2
Введите элемент для добавления: 7

!!! Поймано исключение: Ошибка: превышен максимальный размер множества (operator+)
Программа завершена из-за исключения.
```

Рис. 4 — Исключение 4.

```
Текущие множества:
A: {  }
B: {  }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 4

!!! Поймано исключение: Ошибка: попытка удалить элемент из пустого множества (operato
)
Программа завершена из-за исключения.
```

Рис. 5 — Исключение 5.

```
Текущие множества:
A: {  }
B: {  }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 3

!!! Поймано исключение: Ошибка: попытка удалить элемент из пустого множества (operato
Программа завершена из-за исключения.
```

Рис. 6 — Исключение 6.

```
A: { 1, 2 }
B: { 5, 6 }

Выберите действие:
1. operator[]  (доступ по индексу из A)
2. operator+   (добавить элемент к A, создавая новое множество)
3. operator-- (префиксный) - удалить последний элемент из A
4. operator-- (постфиксный) - удалить последний элемент из A
5. operator-   (удалить элемент по значению из A)
6. operator*   (пересечение A и B)
7. operator<   (проверить, является ли A строгим подмножеством B)
8. Выход из теста множеств
Ваш выбор: 8

=== Перевод числа из 13-ричной системы в 8-ричную ===
Введите число в 13-ричной системе: 13.a1
Результат в 8-ричной системе: 20.6147003017
Программа успешно завершена.
```

Рис. 7 — Перевод из 13 системы счисления в 8.

Листинг

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
#include <cmath>
#include <cctype>
#include <stdexcept>
#include <limits>

using namespace std;

template<typename T>
class Set {
private:
vector<T> elements;
size_t maxSize;

public:
Set(size_t max_size = 100) : maxSize(max_size) {}

Set(size_t max_size, initializer_list<T> list) : maxSize(max_size) {
for (const T& item : list) {
add(item);
}
}

void add(const T& item) {
if (!contains(item)) {
if (elements.size() + 1 > maxSize) {
throw overflow_error("Ошибка: превышен максимальный размер множества (add)");
}
elements.push_back(item);
}
}

bool contains(const T& item) const {
return find(elements.begin(), elements.end(), item) != elements.end();
}

void remove(const T& item) {
auto it = find(elements.begin(), elements.end(), item);
if (it != elements.end()) {
elements.erase(it);
}
}
```

```
size_t size() const {
return elements.size();
}

bool empty() const {
return elements.empty();
}

const T& operator[](size_t index) const {
if (index >= elements.size()) {
throw out_of_range("Ошибка: индекс вне допустимого диапазона (operator[])");
}
return elements[index];
}

Set operator+(const T& item) const {
Set result = *this;
if (!result.contains(item)) {
if (result.size() + 1 > result.maxSize) {
throw overflow_error("Ошибка: превышен максимальный размер множества
(operator+)");
}
result.elements.push_back(item);
}
return result;
}

Set& operator--() {
if (empty()) {
throw underflow_error("Ошибка: попытка удалить элемент из пустого множества
(operator--)");
}
elements.pop_back();
return *this;
}

Set operator--(int) {
if (empty()) {
throw underflow_error("Ошибка: попытка удалить элемент из пустого множества
(operator-- int)");
}
Set temp = *this;
elements.pop_back();
return temp;
}

Set operator-(const T& item) const {
Set result = *this;
```

```
result.remove(item);
return result;
}

Set operator*(const Set& other) const {
Set result(maxSize);
for (const T& item : elements) {
if (other.contains(item)) {
result.add(item);
}
}
return result;
}

bool operator<(const Set& other) const {
for (const T& item : elements) {
if (!other.contains(item)) {
return false;
}
}
return size() < other.size();
}

friend ostream& operator<<(ostream& os, const Set& set) {
os << "{ ";
for (size_t i = 0; i < set.elements.size(); ++i) {
os << set.elements[i];
if (i + 1 < set.elements.size()) os << ", ";
}
os << " }";
return os;
}
};

template<typename T>
Set<T> inputSet(const string& prompt, size_t max_size) {
cout << prompt;
int n;
cin >> n;
if (cin.fail()) {
cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
throw invalid_argument("Ошибка ввода: размер должен быть целым числом");
}
Set<T> set(max_size);
cout << "Введите " << n << " элементов(а): ";
for (int i = 0; i < n; ++i) {
T item;
```

```
cin >> item;
set.add(item);
}
return set;
}

int charToValue13(char ch) {
if (ch >= '0' && ch <= '9') return ch - '0';
if (ch >= 'A' && ch <= 'C') return ch - 'A' + 10;
if (ch >= 'a' && ch <= 'c') return ch - 'a' + 10;
throw invalid_argument("Недопустимый символ в 13-ричном числе");
}

double fromBase13(const string& num) {
if (num.empty()) throw invalid_argument("Пустая строка");

size_t dotPos = num.find('.');
string intPart = (dotPos == string::npos) ? num : num.substr(0, dotPos);
string fracPart = (dotPos == string::npos) ? "" : num.substr(dotPos + 1);

double result = 0.0;

for (char ch : intPart) {
int val = charToValue13(ch);
result = result * 13 + val;
}

double factor = 1.0 / 13;
for (char ch : fracPart) {
int val = charToValue13(ch);
result += val * factor;
factor /= 13;
}

return result;
}

string toBase8(double num, int precision = 10) {
bool negative = false;
if (num < 0) {
negative = true;
num = -num;
}

long long intPart = static_cast<long long>(num);
double fracPart = num - intPart;

string intStr;
```

```
if (intPart == 0) {
intStr = "0";
} else {
while (intPart > 0) {
int digit = intPart % 8;
intStr.push_back('0' + digit);
intPart /= 8;
}
reverse(intStr.begin(), intStr.end());
}

string fracStr;
for (int i = 0; i < precision && fracPart > 1e-10; ++i) {
fracPart *= 8;
int digit = static_cast<int>(fracPart);
fracStr.push_back('0' + digit);
fracPart -= digit;
}

string result = (negative ? "-" : "") + intStr;
if (!fracStr.empty()) {
result += "." + fracStr;
}
return result;
}

string convert13to8(const string& input) {
double decimal = fromBase13(input);
return toBase8(decimal);
}

int main() {
cout << "=== Интерактивная проверка исключений класса Set ===\n";
size_t maxSize;
cout << "Введите максимальный размер множеств: ";
cin >> maxSize;
if (cin.fail() || maxSize == 0) {
cerr << "Некорректный ввод. Программа завершена.\n";
return 1;
}

Set<int> userSetA, userSetB;
try {
userSetA = inputSet<int>("Введите размер первого множества: ", maxSize);
userSetB = inputSet<int>("Введите размер второго множества: ", maxSize);
} catch (const exception& e) {
cerr << "Исключение при вводе множества: " << e.what() << endl;
return 1;
```

```
}

cout << "\n--- Меню операций (любое исключение завершит программу) ---\n";
try {
while (true) {
cout << "\nТекущие множества:\n";
cout << "A: " << userSetA << endl;
cout << "B: " << userSetB << endl;
cout << "\nВыберите действие:\n";
cout << "1. operator[] (доступ по индексу из A)\n";
cout << "2. operator+ (добавить элемент к A, создавая новое множество)\n";
cout << "3. operator-- (префиксный) – удалить последний элемент из A\n";
cout << "4. operator-- (постфиксный) – удалить последний элемент из A\n";
cout << "5. operator- (удалить элемент по значению из A)\n";
cout << "6. operator* (пересечение A и B)\n";
cout << "7. operator< (проверить, является ли A строгим подмножеством B)\n";
cout << "8. Выход из теста множеств\n";
cout << "Ваш выбор: ";
int choice;
cin >> choice;
if (choice == 8) break;
switch (choice) {
case 1: {
size_t index;
cout << "Введите индекс: ";
cin >> index;
int val = userSetA[index];
cout << "Элемент A[" << index << "] = " << val << endl;
break;
}
case 2: {
int item;
cout << "Введите элемент для добавления: ";
cin >> item;
Set<int> newSet = userSetA + item;
cout << "Результат A + " << item << " = " << newSet << endl;
break;
}
case 3: {
--userSetA;
cout << "После --A: " << userSetA << endl;
break;
}
case 4: {
userSetA--;
cout << "После A--: " << userSetA << endl;
break;
}
```

```
case 5: {
int item;
cout << "Введите элемент для удаления: ";
cin >> item;
Set<int> newSet = userSetA - item;
cout << "Результат A - " << item << " = " << newSet << endl;
break;
}
case 6: {
Set<int> inter = userSetA * userSetB;
cout << "Пересечение A * B = " << inter << endl;
break;
}
case 7: {
bool res = userSetA < userSetB;
cout << "A < B ? " << (res ? "да" : "нет") << endl;
break;
}
default:
cout << "Неверный выбор. Попробуйте снова.\n";
}
}
} catch (const exception& e) {
cerr << "\n!!! Поймано исключение: " << e.what() << endl;
cerr << "Программа завершена из-за исключения.\n";
return 1;
}

cout << "\n=== Перевод числа из 13-ричной системы в 8-ричную ===\n";
cout << "Введите число в 13-ричной системе: ";
string input;
cin.ignore();
getline(cin, input);

try {
string result = convert13to8(input);
cout << "Результат в 8-ричной системе: " << result << endl;
} catch (const invalid_argument& e) {
cerr << "Ошибка при переводе: " << e.what() << endl;
return 1;
} catch (const exception& e) {
cerr << "Неизвестная ошибка: " << e.what() << endl;
return 1;
}

cout << "Программа успешно завершена.\n";
return 0;
}
```